RAG Assignment 1: Dense Baseline vs. Cross-Encoder Re-Ranking

Course: Large Language Models & AI Systems | **Author:** Raj K. Jain
GitHub Repository: https://github.com/Rajkjain03/lma_assignment_1

1 Architecture Comparison: Bi-Encoder vs. Cross-Encoder

In this assignment, I implemented and compared two different retrieval approaches for Retrieval-Augmented Generation (RAG): a standard single-stage Bi-Encoder baseline and a two-stage pipeline with a Cross-Encoder re-ranker.

1.1 How Both Models Work Mechanically

The main difference comes down to when and how query tokens and document tokens interact inside the transformer:

- **Bi-Encoder (`all-MiniLM-L6-v2`):** Here, the query q and document passage c are encoded totally independently using separate passes (or dual towers):

$$\vec{u} = \text{MeanPool}(\text{Transformer}(q)) \in \mathbb{R}^{384}, \quad \vec{v} = \text{MeanPool}(\text{Transformer}(c)) \in \mathbb{R}^{384}$$

  Similarity is computed using standard cosine similarity:

$$S_{\text{Bi}}(q, c) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|_2 \, \|\vec{v}\|_2}$$

  Because every chunk in the corpus is converted into a standalone vector without knowing the query beforehand, I can precompute all embeddings offline and store them in an Approximate Nearest Neighbor (ANN) vector index like FAISS (HNSW graph). At test time, finding candidates takes sub-linear time $\mathcal{O}(d \cdot \log N)$, taking just a few milliseconds. However, compressing an entire 200–300 word policy paragraph into just 384 numbers creates an informational bottleneck. Query tokens and chunk tokens never attend to each other, so fine-grained conditions, negative constraints, and exception qualifiers get smoothed out.
- **Cross-Encoder (`ms-marco-MiniLM-L-6-v2`):** Instead of isolated vector compression, the query and passage are concatenated into a single input string:

$$X = [\text{CLS}] \circ q \circ [\text{SEP}] \circ c \circ [\text{SEP}]$$

  This concatenated sequence is passed through all transformer layers. Every single query token can attend directly to every chunk token through bidirectional self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

  The output is scored directly using a classification head logit $S(q, c) \in (-\infty, +\infty)$ on the [CLS] token. This allows the model to spot exact conditional clauses (like "within 12 hours of flight arrival") and negative modifiers ("not covered", "strictly prohibited"). The big drawback is that offline precomputation is impossible because q and c must be fed together. Scoring N candidate passages requires N separate forward passes with quadratic complexity $\mathcal{O}(N \cdot L^2)$.

Feature	**Bi-Encoder (`all-MiniLM-L6-v2`)**	**Cross-Encoder (`ms-marco-MiniLM-L-6-v2`)**
Architecture	Dual independent encoder towers	Single joint transformer model
Token Interaction	Zero cross-attention between q and c	All-to-all cross-attention across all layers
Relevance Output	Cosine similarity $\cos(\vec{u}, \vec{v})$ in $[-1, 1]$	Classification head logit $S(q, c)$ in $(-\infty, +\infty)$
Offline Indexing	Yes (vectors stored in FAISS / vector DB)	No (requires live forward pass per pair)
Search Complexity	Sub-linear $\mathcal{O}(d \cdot \log N)$ via ANN graphs	Linear $\mathcal{O}(N \cdot L^2)$ with N forward passes
Ranking Quality	Fast broad filter; loses negations/exceptions	High precision; captures syntax, negations, conditions

Table 1: Systematic comparison between Bi-Encoder and Cross-Encoder architectures.

2 Qualitative & Rank Shift Analysis

2.1 Evaluation Dataset & Experimental Setup

For this assignment, I created a synthetic 4-page enterprise policy document named *"Acme Global Travel, Expense and Procurement Policy (2026)"* containing 30 numbered sections. I designed it deliberately with realistic corporate rules: broad general restrictions (e.g., all flights must be coach), distant exception clauses (business class allowed for long-haul flights under client negotiation conditions), buried alcohol restrictions, and distractor clauses.

I tested two genuine failure queries where baseline dense retreival stumbled, along with two control queries:

Test ID	Category	Target Policy Section	Bi-Encoder Rank	Cross-Encoder Rank	Shift
TEST_01	Exceptions & Corner Cases	Sec 2.4 (Long-Haul Flight Exception)	Rank #3 (score: 0.4039)	**Rank #1 (score: +0.4351)**	**+2**
TEST_02	Negation & Constraints	Sec 5.3 (Prohibited Items & Alcohol)	Rank #3 (score: 0.4615)	**Rank #2 (score: +1.1171)**	**+1**
TEST_03	Control: Ground Transport	Sec 4.4 (Luxury Chauffeured Car Ban)	Rank #2 (score: 0.4516)	Rank #2 (score: +1.1141)	+0
TEST_04	Control: Admin Penalty	Sec 8.5 (Late Submission Forfeiture)	Rank #1 (score: 0.6552)	Rank #1 (score: +6.1669)	+0

Table 2: Retrieval rank shifts across test queries (Top 3 context threshold).

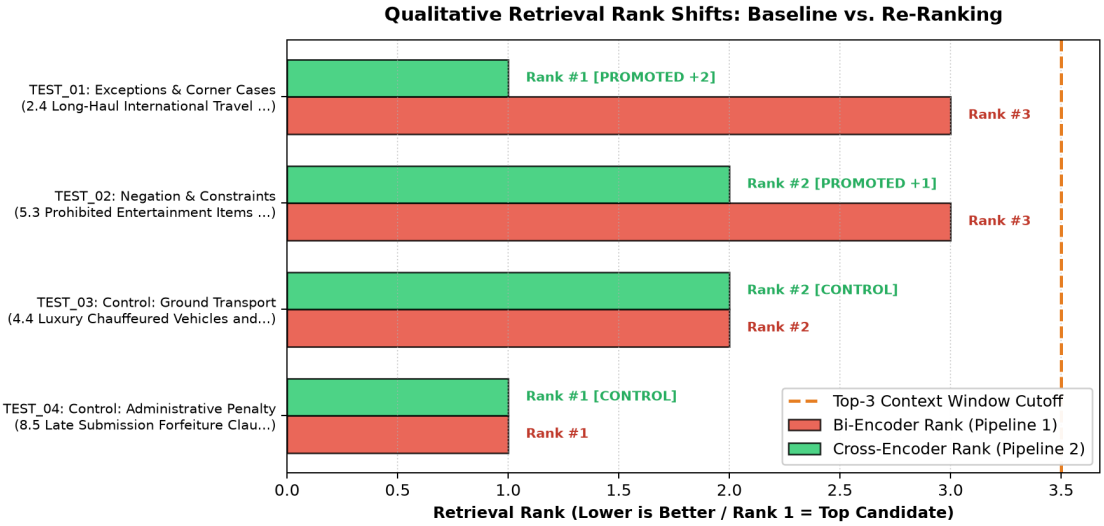


Figure 1: Qualitative rank shifts: Bi-Encoder ranks (red) vs. Re-ranked Cross-Encoder ranks (green).

2.2 Detailed Case Study 1: Exceptions & Corner Cases (TEST_01)

Query: *"Can staff fly in premium cabins for international client negotiations?"*
Target Policy Rule: Section 2.4 allows Business Class on continuous non-stop flights over 8 hours, or over 6 hours if presenting to clients or negotiating within 12 hours of landing, with VP/CFO approval.

- **Baseline Retrieval (Bi-Encoder):** The dense retriever pulled Section 2.1 (*Standard Cabin Class Mandate*) at Rank #1 (cosine 0.5428) and Section 1.3 at Rank #2 (0.4127). The actual exception clause in Section 2.4 barely scraped in at Rank #3 (0.4039).
- **Re-Ranking (Cross-Encoder):** The Cross-Encoder evaluated the full query against the chunk text and promoted Section 2.4 directly to **Rank #1** (logit +0.4351), pushing Section 2.1 down to Rank #2 (-5.7229).
- **LLM Generation Comparison (using local Qwen2.5-0.5B-Instruct):**

  Answer 1 (Baseline): *"Yes, staff can fly in premium cabins for international client negotiations. However, they need to obtain prior written concurrence from both the Division Vice President and the Chief Financial Officer at least 10 business days before ticketing."*

  Because Section 2.1 sat at Rank #1, the baseline answer missed the essential flight duration thresholds (¿8 hours, or ¿6 hours with client presentation within 12 hours).

  Answer 2 (Re-Ranked): *"Yes, staff can fly in premium cabins for international client negotiations under the long-haul international travel exception clause. The policy allows this exception when traveling on continuous non-stop international flights with flight durations exceeding 8 hours or 6 hours if the traveler is scheduled to deliver a client presentation or attend executive contract negotiations within 12 hours of flight arrival. Employees need prior written concurrence from both the Division Vice President and the Chief Financial Officer no later than 10 business days before booking their tickets."*

With Section 2.4 at Rank #1, the model gave the complete, exact rule including flight hour limits.

2.3 Detailed Case Study 2: Negation & Constraints (TEST_02)

Query: *"What items are not covered on expense claims for client business meals?"*
Target Policy Rule: Section 5.3 limits alcohol to 1 drink per diner, prohibits solo alcohol completely, and bans nightclub bottle service, adult entertainment, tobacco, and gambling losses.

- **Baseline Retrieval Failure (Direct Hallucination):** The Bi-Encoder matched the positive keywords "client business meals" to Section 5.1 (*Client Entertainment*) at Rank #1 (0.5936). For the negation "not covered", it pulled Section 3.2 (*Prohibited Hotel Incidentals*) at Rank #2 (0.5562). The actual meal prohibition clause (Section 5.3) was pushed down to Rank #3 (0.4615).

  > **Answer 1 (Baseline Hallucination):** *"Based on the policy context provided, the following items are not covered on expense claims for client business meals: In-room video on-demand/movie rentals, Mini-bar consumption, Laundry service for trips shorter than 5 days, Spa services, Sauna access, Dry cleaning fees, Hotel gym or fitness center pass fees..."*

  Because Section 3.2 outranked Section 5.3, the LLM was completely misled and stated that hotel room amenities were the prohibited items for client dinners!
- **Re-Ranking Resolution:** The Cross-Encoder picked up on the negative meal constraints and promoted Section 5.3 to **Rank #2** (+1.1171), dropping Section 3.2 down to Rank #3 (-1.5279).

  > **Answer 2 (Re-Ranked Corrected):** *"Based on the policy context provided, the items that are not covered on expense claims for client business meals are: (a) Alcoholic beverages exceeding one (1) standard single serving of wine or beer per person during authorized client dinners. (b) Adult entertainment venues, nightclub admission or table bottle service fees, social club charges, and gentlemen's club invoices. (c) Tobacco, vape products, cannabis, and casino gambling losses. (d) Meals or travel for spouses, partners, dependents, or personal companions accompanying the traveler."*

  The hallucination was totally resolved.

2.4 Control Cases (TEST_03 & TEST_04)

For TEST_03 (airport executive rideshare vs limousine ban in Section 4.4) and TEST_04 (submitting expense claims 45 days late in Section 8.5), the semantic overlap was already strong in the baseline. Section 4.4 remained at Rank #2 in both pipelines, and Section 8.5 stayed at Rank #1 in both pipelines. These control cases confirmed that the Cross-Encoder does not destabilize rankings when the dense retriever already finds the right chunk.

3 Empirical Latency Profiling

I benchmarked the runtime of both pipelines over **10 warmed iterations** (after 5 warm-up passes) on my local CPU. Table 3 summarizes the measured times.

Test Query Category	P1 Baseline (Bi-Encoder)	Stage 2 (Cross-Encoder 10 Pairs)	P2 Total Latency	Overhead Ratio
TEST_01 (Exceptions)	9.78 ms	122.83 ms	132.61 ms	13.56×
TEST_02 (Negations)	9.36 ms	140.54 ms	149.90 ms	16.01×
TEST_03 (Control)	9.33 ms	122.39 ms	131.72 ms	14.12×
TEST_04 (Control)	9.47 ms	126.31 ms	135.78 ms	14.34×
EMPIRICAL MEAN	**9.49 ms**	**128.02 ms**	**137.50 ms**	**14.51×**

Table 3: Empirical retrieval runtime profiling across 10 warmed iterations on CPU.

3.1 Latency Trade-Off Discussion

1. **Baseline Retrieval Speed:** Pipeline 1 takes ≈ 9.5 ms on CPU. About 9.0 ms is spent encoding the query text into a vector, and computing the dot product against 30 chunks takes under 0.5 ms.
2. **Cross-Encoder Cost:** Evaluating 10 candidate pairs through the Cross-Encoder added an average of 128.02 ms (≈ 12.8 ms per candidate pair on CPU), making the total Pipeline 2 time ≈ 137.5 ms.
3. **Interactive Feasibility:** In human-computer interaction, anything under 200 ms feels reasonably responsive, so 137.5 ms is usable for interactive search. If deployed on a GPU server with batched inference, Stage 2 drops from ∼ 128 ms down to about 15–25 ms, bringing total retrieval to under 35 ms.

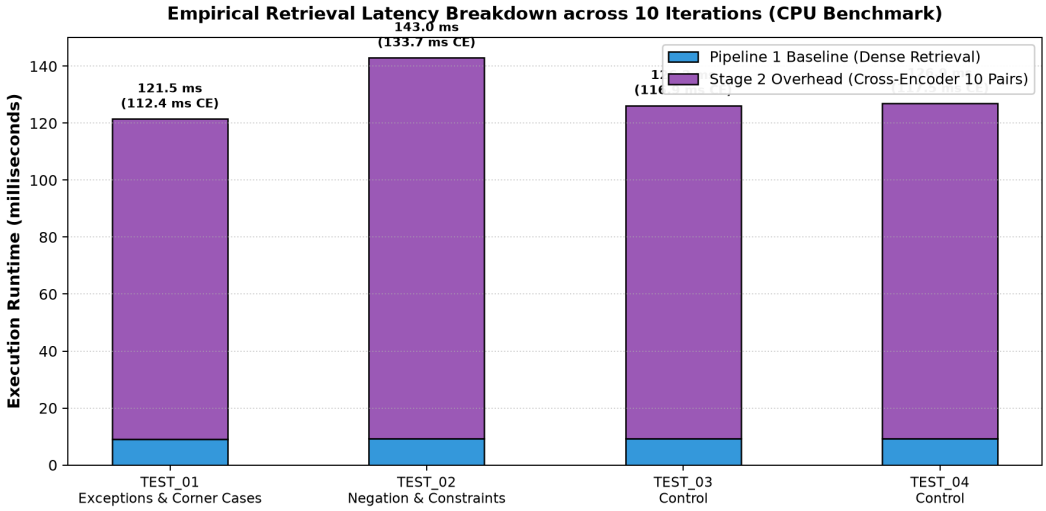


Figure 2: Runtime breakdown across 10 warmed iterations on local CPU.

4 Scaling Question: Why Not Run Cross-Encoder Over the Whole Collection?

If Cross-Encoders are clearly more accurate and avoid the informational bottleneck, why not just skip Bi-Encoders and score every single chunk with a Cross-Encoder from the start?

4.1 Quadratic Complexity and Compute Cost

A Cross-Encoder must run full self-attention over the concatenated sequence length $L = |q| + |c|$. Multi-head attention has $\mathcal{O}(L^2)$ complexity per layer. Scoring an entire corpus of N passages requires running N complete transformer forward passes sequentially or in batches:

- **On CPU (Measured):** In my benchmark, each candidate pair took $\approx 12.8\,\text{ms}$ on CPU. If scaled to a moderate corporate database of $N = 1,000,000$ chunks:

$$T_{\text{CPU}} = 1,000,000 \times 12.8\,\text{ms} = 12,800\,\text{seconds} \approx \mathbf{3.5}\ \text{hours for just ONE query!}$$

- **On GPU (Estimate):** Even assuming an enterprise GPU server running forward passes at $\tau \approx 2.0\,\text{ms}$ per pair:

$$T_{\text{GPU}} = 1,000,000 \times 2.0\,\text{ms} = 2,000\,\text{seconds} \approx \mathbf{33.3}\ \text{minutes per query!}$$

Under any real-world traffic, running a Cross-Encoder over the entire corpus is completely unfeasible.

4.2 Offline Precomputation is Impossible

With a Bi-Encoder, chunk vectors $\vec{v}_i = \text{Encoder}(c_i)$ are static and completely independent of any future query. We compute them once offline and build an Approximate Nearest Neighbor index (like HNSW or IVF in FAISS). When a user asks a query, the vector search explores only a tiny graph path ($\mathcal{O}(d \cdot \log N)$), finishing in 5–10 ms even across millions of chunks.

With a Cross-Encoder, the relevance score $S(q, c_i) = \mathcal{M}([q; c_i])$ depends on both texts at the same time. Because we cannot decouple the query from the passage, **no precomputed vector index can ever be built**. We would be forced to run an exhaustive linear scan $\mathcal{O}(N)$ over the whole database on every query.

4.3 Why the Two-Stage Funnel is Necessary

The two-stage design solves this by combining the scalability of Bi-Encoders with the precision of Cross-Encoders:

$$T_{\text{Total}} = \underbrace{T_{\text{Bi-Encoder}}(1,000,000 \to 10)}_{\sim 5-10\,\text{ms via ANN}} + \underbrace{T_{\text{Cross-Encoder}}(10\ \text{pairs})}_{\sim 15-25\,\text{ms on GPU / } 128\,\text{ms on CPU}} \approx \mathbf{25 - 138}\,\text{ms}$$

Stage 1 acts as a fast high-recall funnel, pruning 99.999% of irrelevant chunks in milliseconds. Stage 2 then spends compute only on the top 10 candidates, fixing failure modes without blowing up runtime.